Inference-Time Repair for Robust Tool Calling Under Interface Drift

Anonymous Authors

Draft prepared for NGEN-AI 2026

Abstract. Tool-calling language models rely heavily on surface-form tool interfaces, including descriptions, parameter names, and candidate tool sets. In realistic deployments, these interfaces drift after release even when the underlying tool semantics remain unchanged. We study this problem as *interface drift* and evaluate a lightweight training-free mitigation, SchemaShield-Lite, built from canonical tool cards, validation, and one-shot repair. We evaluate four open models across two benchmarks (DICE-BENCH at n=300 and BFCL at n=200) under three drift perturbations applied individually and in combination. On DICE-BENCH with Qwen3.5-9B, accuracy drops from 0.750 to 0.713 under drift, while SchemaShield-Lite recovers performance to 0.827 and fixes 17/18 drift-induced failures with zero harms. Across all four models, repair recovers drift-induced failures with zero harms. A naive-retry baseline scores 0.700 on DICE (worse than drifted) and 0.650 on BFCL, demonstrating that the gains come from structured validation feedback rather than from a second attempt. Drift-type ablation shows that candidate-set drift dominates: description and schema drift alone cause < 1.5 point degradations on modern models, while distractor tools cause most of the observed harm. A component ablation reveals that the canonical tool card is not the load-bearing component: validation feedback combined with forced-tool repair accounts for essentially all of the recovery. On BFCL with Llama-4-Scout, SchemaShield-Lite improves accuracy from 0.685 to 0.855, a 17.0 point lift over drifted performance.

Keywords: tool calling · language models · robustness · interface drift · repair

1 Introduction

Modern language-model agents increasingly depend on external tools for retrieval, execution, planning, and actuation. In practice, however, tool use is mediated entirely through text: tool descriptions, parameter schemas, enum values, and candidate tool lists. Recent work shows that these interface details strongly affect tool selection and argument generation even when the tool semantics are unchanged [2, 1]. This creates a deployment problem: tool interfaces drift over time as developers rename parameters, revise descriptions, add examples, or expand candidate tool sets.

We study this problem as *interface drift*: benign, semantics-preserving changes to tool interfaces that occur after deployment. Our goal is not to train a new tool-calling model, but to quantify drift sensitivity and test whether a compact inference-time repair layer can recover reliability without retraining.

We instantiate drift with three realistic perturbations: (i) stale description examples that mention legacy field names, (ii) schema renaming, and (iii) distractor tools. We evaluate four open models (Qwen3.5-9B, Qwen3.5-35B-A3B, Llama-4-Scout, and Mistral-Small-24B) on a 300-example DICE-BENCH slice [3] and a 200-example BFCL slice [5]. We apply SchemaShield-Lite, a deterministic inference-time stack combining canonical tool rendering, validation, and one-shot forced-tool repair.

This paper makes four empirical contributions that substantially revise the narrative of prior interface-drift work:

1. **Inference-time repair recovers drift-induced failures across models and benchmarks with zero harms.** On DICE-BENCH, SchemaShield-Lite recovers 17/18 drift-induced failures on Qwen3.5-9B and achieves zero repair harms across all four models. On BFCL, repair improves Llama-4-Scout by 17 points and Qwen3.5-9B by 15.5 points over drifted performance.
2. **A naive-retry baseline underperforms even the drifted model.** On DICE-300, naive retry scores 0.700 versus drifted 0.713; on BFCL, 0.650 vs 0.640. This rules out the trivial explanation that repair works simply by giving the model a second attempt.
3. **Candidate-set drift is the dominant failure mode on modern models.** Under description-only drift or schema-only drift, Qwen3.5-9B accuracy drops by just 0.013 and 0.007 points respectively. Adding distractor tools causes a 0.047 drop, matching the full combined drift.
4. **Validation feedback, not the canonical tool card, is the load-bearing component of SchemaShield-Lite.** A component ablation shows that *validation-plus-retry without the canonical card* recovers 16 of 17 drift-induced failures, nearly matching the full pipeline. The canonical card alone never triggers repair and recovers nothing.

2 Related Work

Recent work has established that tool interfaces are not neutral wrappers around tool use; they are part of the learning and inference problem itself. Guo et al. [2] show that rewriting tool descriptions can substantially improve agent reliability, especially when candidate tool sets are large. Faghih et al. [1] show the opposite direction: small edits to tool descriptions can dramatically shift tool preference, exposing a fragility in description-mediated tool use.

Several recent papers study robustness and recovery in tool-use systems. Zhang et al. [6] show that execution-error recovery remains a major weakness, particularly for smaller models, and propose RL-based training to improve it. Our setting is narrower and more deployment-oriented: we target inference-time robustness under interface drift rather than post-training optimization.

Benchmark work has also highlighted that realistic function calling is still far from solved. ComplexFuncBench introduces multi-step and constrained long-context function calling [7], while DICE-BENCH focuses on multi-round, multi-party dialogues with tool-related information dispersed across conversation turns [3]. We evaluate on both DICE-BENCH and the Berkeley Function Calling Leaderboard (BFCL) [5] because they capture complementary settings: DICE-BENCH exercises conversational multi-turn dialogues, while BFCL tests single-turn tool calls across diverse APIs.

Compared with dataset-generation work such as APIGen [4], our objective is different. We do not create new large-scale tool-calling training data. Instead, we evaluate whether a lightweight repair layer can stabilize existing models against a specific class of post-deployment failures.

3 Problem Formulation

Let a tool be represented by an interface $I = (d, s, C)$, where d is the natural-language description, s is the argument schema, and C is the candidate tool set shown to the model. The underlying tool semantics are assumed fixed. An *interface drift* operator Δ maps I to a new interface $\Delta(I)$ that preserves tool semantics but perturbs surface form.

We study three drift types:

- **Description drift**: add stale or legacy examples to the description that refer to older field names.
- **Schema drift**: rename parameters while preserving semantics.
- **Candidate-set drift**: add distractor tools with overlapping descriptions.

Given user context x, tool interface I, and model M, the base system outputs a tool call $y = M(x, I)$. We evaluate three stages:

1. **Original**: inference on the original tool interface.
2. **Drifted**: inference on the drifted interface $\Delta(I)$.
3. **Repaired**: inference-time repair of an invalid or incomplete drifted call.

Our central question is whether an inference-time repair layer can recover the performance lost from Original to Drifted without harming examples that were already correct.

4 Method

4.1 SchemaShield-Lite

SchemaShield-Lite is a lightweight inference-time robustness layer with three parts:

Canonical tool card. Each tool is rendered into a compact canonical form containing the tool name, description, required fields, and field-level type information. This normalizes interface presentation across tools and emphasizes current field names.

Validation. After the base model emits a tool call, a validator checks tool name, required fields, and argument structure. Validation produces a structured error report rather than a binary failure signal.

One-shot repair. If the drifted call is invalid, the model receives a repair prompt containing the task context, canonical tool card, invalid call, and validation errors. Repair uses a single constrained call. In the strongest configuration we use *forced-tool repair*: the repair step is constrained to the intended tool when provider support allows it, with JSON fallback if the provider does not support named tool choice.

As our component ablation in Section 6.4 shows, it is the combination of structured validation errors and forced-tool repair that does the heavy lifting. The canonical card is useful at the initial prediction step as drift-prevention prompting, but contributes negligibly to repair-stage recovery.

4.2 Repair Prompting Details

Two implementation details turned out to matter materially:

- The repair prompt explicitly instructs the model to infer all required fields from the dialogue even when the invalid call is empty.
- The repair stage uses a larger token budget than the base prediction stage. Raising repair capacity from the earlier default to 512 tokens fixed truncated repair outputs on longer DICE dialogues.

5 Experimental Setup

5.1 Benchmarks and Data Slices

We evaluate on two benchmarks. For DICE-BENCH [3], we use a 300-example subset drawn from single-function round-1 dialogues (allowing duplicate functions to reach sufficient scale). For BFCL [5], we use a 200-example subset drawn in equal parts from four categories: simple Python, multiple-function, live-simple, and live-multiple.

5.2 Models

We evaluate four open instruction-tuned models accessed via OpenRouter:

- **Qwen3.5-9B**: our primary model.
- **Qwen3.5-35B-A3B**: larger MoE variant.
- **Llama-4-Scout**: Meta's 2025 open model.
- **Mistral-Small-3.2-24B**: recent Mistral dense model.

All experiments use temperature 0.0 for deterministic-style evaluation.

Table 1. Main results on DICE-300 and BFCL-200. OC = originally-correct count; Rec = recoveries; Harms = repair harms on originally-correct slice.

Benchmark	Model	n	Orig.	Drift.	Repair	OC	Rec/Miss	Harms	Trig%
DICE-300	Qwen3.5-9B	300	0.750	0.713	**0.827**	225	17/18	0	21.0
	Qwen3.5-35B-A3B	300	0.773	0.713	**0.820**	232	20/25	0	21.3
	Llama-4-Scout	300	0.707	0.717	**0.807**	212	12/16	0	13.3
	Mistral-24B	300	0.740	0.777	**0.827**	222	8/12	0	9.0
BFCL-200	Qwen3.5-9B	200	0.790	0.640	**0.795**	158	23/35	0	31.0
	Llama-4-Scout	200	0.535	0.685	**0.855**	107	9/15	0	21.0

5.3 Drift Configurations

For DICE main runs, we use *mild* severity: two description modes (`legacy_example`, `marketing`), two schema modes (`rename_parameters`, `reorder_parameters`), and one distractor tool. For BFCL we use *medium* severity: three description modes, three schema modes, and three distractors, reflecting a more aggressive drift scenario.

5.4 Metrics

We report:

- **Original score**: accuracy on the unmodified interface.
- **Drifted score**: accuracy under interface drift.
- **Repaired score**: accuracy after one-shot repair.
- **Naive retry score**: accuracy under a second identical drifted call (no canonical card, no validation feedback). This baseline isolates the value of simply retrying.
- **Originally-correct slice**: the subset of examples that the model solves correctly before drift. This slice isolates *true drift harm* from baseline errors.
- **Repair recoveries**: drift-induced failures recovered by repair.
- **Repair harms**: drifted-correct examples broken by repair.
- **Repair trigger rate**: fraction of examples for which the validator invokes the repair step.

6 Results

6.1 Main Results Across Models and Benchmarks

Table 1 summarizes results across all four models on DICE-300 and both models evaluated on BFCL-200. Across every model and both benchmarks, SchemaShield-Lite improves over the drifted baseline with *zero repair harms*.

The BFCL-200 configuration uses medium-severity drift and shows substantially larger effects than DICE mild drift. On Qwen3.5-9B, drift drops accuracy

Table 2. Naive retry vs SchemaShield-Lite on Qwen3.5-9B.

Benchmark	Drifted	Naive Retry	SchemaShield-Lite	Δ over Drifted
DICE-300	0.713	0.700	**0.827**	+0.114
BFCL-200	0.640	0.650	**0.795**	+0.155

Table 3. Drift-type ablation on DICE-300 with Qwen3.5-9B.

Drift type	Orig.	Drift.	Repair	Drift Drop
Description only	0.760	0.747	0.813	−0.013
Schema only	0.767	0.760	0.830	−0.007
Candidate-set only	0.757	0.710	0.830	−0.047
All combined	0.763	0.713	0.823	−0.050

by 15.0 points (0.790 → 0.640) and repair fully recovers (0.795). On Llama-4-Scout, repair improves accuracy by 17.0 points over drifted and 32.0 points over original.

6.2 Naive Retry Is Not a Viable Baseline

A first-order alternative explanation is that repair works simply because the model gets a second attempt. We test this by running a *naive retry* call that uses the same drifted tools but without the canonical card or the validation error report. Table 2 shows that naive retry underperforms even the drifted baseline: 0.700 vs 0.713 on DICE and 0.650 vs 0.640 on BFCL. In contrast, SchemaShield-Lite achieves 0.827 on DICE and 0.795 on BFCL. The gap rules out retry-alone as the mechanism and localizes the improvement to structured validation feedback combined with forced-tool repair.

6.3 Drift-Type Ablation: Candidate-Set Drift Dominates

Table 3 decomposes the drift pipeline into its three components and evaluates each in isolation at DICE-300 on Qwen3.5-9B. Description drift alone causes only a 0.013 point accuracy drop. Schema drift alone causes only 0.007. By contrast, candidate-set drift (adding a distractor tool) causes a 0.047 drop—matching the full combined drift scenario (0.050).

This result qualifies a common framing in the interface-drift literature. For modern instruction-tuned tool-calling models, minor description rewording and semantics-preserving parameter renames do not meaningfully degrade accuracy—the models are evidently robust to this class of surface perturbation. The realistic failure mode is *candidate-set confusion*: when a distractor tool with an overlapping description is added to the candidate set, tool selection degrades. This has implications for deployment-time monitoring: teams should prioritize tracking tool-catalog growth over monitoring description or schema revisions.

Table 4. Component ablation on DICE-300 with Qwen3.5-9B.

Variant	Drift.	Repair	Rec/Miss	Harms	Trig%
Canonical card only	0.770	0.770	0/9	0	0.0
Validation + retry (no card)	0.730	0.830	16/17	0	20.0
Full SchemaShield-Lite	0.713	**0.827**	17/18	0	21.0

6.4 Component Ablation: Validation, Not Canonical Card, Drives Recovery

Table 4 evaluates three variants of SchemaShield-Lite on DICE-300 with Qwen3.5-9B: (i) canonical card only, where the card is prepended to the initial prompt but no validation/repair is applied; (ii) validation-plus-retry, which uses validation errors in a repair prompt but omits the canonical tool card from the repair context; and (iii) full SchemaShield-Lite (card + validation + repair).

Three observations. First, prepending the canonical card at the initial prompt *reduces* drift damage (0.770 drifted vs 0.713 for the base drifted run), indicating that the card is useful as drift-prevention prompting—it helps the model resist drift at generation time. Second, however, canonical card alone never triggers the validator and therefore recovers zero drift-induced failures. Third, validation-plus-retry without the canonical card achieves 0.830 repaired accuracy and 16/17 recoveries, essentially matching the full pipeline (0.827, 17/18).

The practical implication is that the load-bearing component of SchemaShield-Lite is the validation error feedback loop combined with forced-tool repair, not the canonical tool card. Teams implementing an inference-time robustness layer can drop the card rendering without meaningfully hurting recovery, or keep the card only at the initial prediction step where it provides drift-prevention benefits.

6.5 Error Analysis

Across the four main DICE runs, all repair harms are zero. However, not every drift miss is recovered. On Qwen3.5-35B, repair recovers 20 of 25 drift misses; on Llama-4-Scout, 12 of 16; on Mistral, 8 of 12. Inspection of the unrecovered cases shows two recurring patterns: (i) the invalid drifted call has an empty arguments object and the model still cannot infer all required fields from the dialogue even with validation feedback, and (ii) for a small number of cases the repair call selects a valid tool but fills in semantically-correct-but-not-identical string values (for example, a paraphrased restaurant name). The former are genuine inference failures; the latter are strict-match artifacts rather than true drift failures.

On the main Qwen3.5-9B run, SchemaShield-Lite fixes 24 examples that the base model got wrong *even before drift*. These are cases where the original call was incomplete or used the wrong tool, and validation feedback plus forced-tool repair surfaces the required fields. This is a welcome side effect: the same mechanism that recovers drift failures also helps patch some base-model errors.

Mistral-24B exhibits an unusual pattern: its drifted score (0.777) is slightly higher than its original score (0.740). The rewording produced by description-mode drift may happen to resemble Mistral's instruction-tuning distribution more closely than the unmodified DICE prompts, which is consistent with the finding that modern models are largely robust to benign description changes.

7 Discussion

Our results support two narrower conclusions than the original SchemaShield-Lite framing suggested. First, inference-time repair is a viable and deployable robustness layer for tool-calling systems under interface drift. It works across model families, across two benchmarks, and adds zero harms at any point. Second, the *mechanism* is validation feedback rather than canonical tool rendering, and the *target* is candidate-set drift rather than description or schema drift.

Together these findings simplify the deployment picture. A practitioner does not need a heavy per-tool canonicalization pipeline; an API-schema validator with a forced-tool repair call is sufficient. And the monitoring story is clearer: the first-order cause of post-deployment tool-calling regressions is likely tool-catalog expansion, not description or parameter rename churn.

8 Limitations

Several limitations remain. First, while we evaluate on two benchmarks and four model families, the BFCL evaluation uses only two models due to API-cost constraints, and BFCL repair does not track token usage for the repair call itself. Second, our drift operators focus on stale descriptions, renamed fields, and distractors; other forms of interface drift (for example, type narrowing, enum replacement, or multi-tool refactors) remain untested. Third, evaluation uses hosted inference and so may incorporate provider-specific tool-calling behavior. Fourth, the "repair > original" effect on Qwen3.5-9B indicates that SchemaShield-Lite is also patching baseline failures—this is useful in deployment but complicates clean attribution of the improvement to drift recovery specifically. Finally, the component ablation finding that the canonical card is non-essential to repair is established on a single benchmark and model; it should be validated on BFCL and additional models before being treated as a general rule.

9 Conclusion

We presented SchemaShield-Lite, a lightweight inference-time approach for robust tool calling under interface drift. Across four open models and two benchmarks, SchemaShield-Lite recovers drift-induced failures with zero harms: 17/18 on Qwen3.5-9B and 20/25 on Qwen3.5-35B, with BFCL gains of up to 17 points over drifted. Three findings qualify the picture. A naive-retry baseline underperforms even the drifted model, so repair gains cannot be explained as a second

attempt. Description and schema drift cause negligible degradation on modern models—candidate-set drift is the dominant failure mode. And validation-plus-retry without the canonical card nearly matches the full pipeline, localizing the load-bearing component to structured validation feedback. Inference-time repair is therefore a practical and deployable complement to model training for tool-mediated language systems, but its mechanism and target are narrower and more specific than prior framing suggested.

References

1. Faghih, K., Wang, W., Cheng, Y., Bharti, S., Sriramanan, G., Balasubramanian, S., Hosseini, P., Feizi, S.: Gaming tool preferences in agentic llms. arXiv preprint arXiv:2505.18135 (2025)
2. Guo, Y., et al.: Learning to rewrite tool descriptions for reliable llm-agent tool use. arXiv preprint arXiv:2602.20426 (2026)
3. Jang, K., Lee, D., Kim, K., Heo, D., Lee, T., Kim, W., Suh, B.: Dice-bench: Evaluating the tool-use capabilities of large language models in multi-round, multi-party dialogues. arXiv preprint arXiv:2506.22853 (2025)
4. Liu, Z., Hoang, T., Zhang, J., Zhu, M., Lan, T., Kokane, S., Tan, J., Yao, W., Liu, Z., Feng, Y., Murthy, R., Yang, L., Savarese, S., Niebles, J.C., Wang, H., Heinecke, S., Xiong, C.: Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. arXiv preprint arXiv:2406.18518 (2024)
5. Yan, F., Mao, H., Ji, C.C.J., Zhang, T., Patil, S.G., Stoica, I., Gonzalez, J.E.: Berkeley function calling leaderboard. `https://gorilla.cs.berkeley.edu/leaderboard.html` (2024)
6. Zhang, Y., et al.: Robust tool use via fission-grpo. arXiv preprint arXiv:2601.15625 (2026)
7. Zhong, Y., et al.: Complexfuncbench: Exploring multi-function calling in large language models. arXiv preprint arXiv:2501.10132 (2025)